PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Architecture and Design Specification

Hospital Management System

Version 1.0

Prepared by

Navya Suresh (PES1UG24CS904)
Daksha S G (PES1UG24CS137)
Dhruv Swatantramath (PES1UG24CS155)
Gnanasagar H (PES1UG24CS171)

Group Number: 11

PES University, Bangalore

Date created: 02-10-2026

Revision History

Name	Date	Reason For Changes	Version
Navya Suresh (PES1UG24CS904)	02-10-2026	Initial draft for Mini-Project Part-1	1.0

Approvals

Role	Name	Signature / Date
Product Owner	Navya Suresh	
QA Lead	Daksha S G	
Developer	Dhruv Swatantramath	
Course Instructor	Dr. Ashok Kumar Patil (UE24CS341A Software Engineering)	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Hospital Management System (HMS) v1.0, so that the implementation in Part-2 follows a shared structure and every requirement of the SRS can be traced to a component.

1.2 Scope

Covers user authentication, patient registration and records, appointment booking, clinical records (consultation, prescription, history), billing, reports and administration. Inpatient, pharmacy and laboratory systems are out of scope (see SRS 1.2).

1.3 Audience

Developers, QA engineers, security reviewers, course instructors and the future maintenance team.

1.4 Definitions

HMS, PHI, API, REST, TLS, RBAC, ADR (Architecture Decision Record), STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege), DAL (Data Access Layer).

2. Document Overview

2.1 How to use this document

Section 3 gives the architecture: goals, component diagram and descriptions, the chosen pattern, technology stack, traceability to requirements and the security architecture. Section 4 gives the design: UML sequence diagrams, API design, data design, error handling, logging and UX. Decisions are recorded as ADRs in Section 3.5.

2.2 Related Documents

HMS SRS v1.0 (requirements and use cases), HMS Test Plan v1.0 (test cases and traceability).

3. Architecture

3.1 Goals and Constraints

- **Goals:** protect patient data (SO-1 to SO-4), 99.5% availability from 06:00 to 22:00 (HMS-NF-004), responses within 2-3 s (HMS-NF-001/002), simple to maintain by students and later by hospital IT.
- **Constraints:** C++17 implementation, open-source libraries only, three-month timeline, DPDP Act 2023 compliance, small team.

3.2 Stakeholders and Concerns

Stakeholder	Concerns
Patients	Privacy of health data, easy booking
Doctors	Speed, correct history and allergy information
Receptionists / Billing	Fast screens, accurate bills
Hospital management / regulators	Compliance, audit trail, reports

Stakeholder	Concerns
Administrator / IT	Easy backup, user management, monitoring
Developers	Modularity, testability

3.3 Component (UML) Diagram

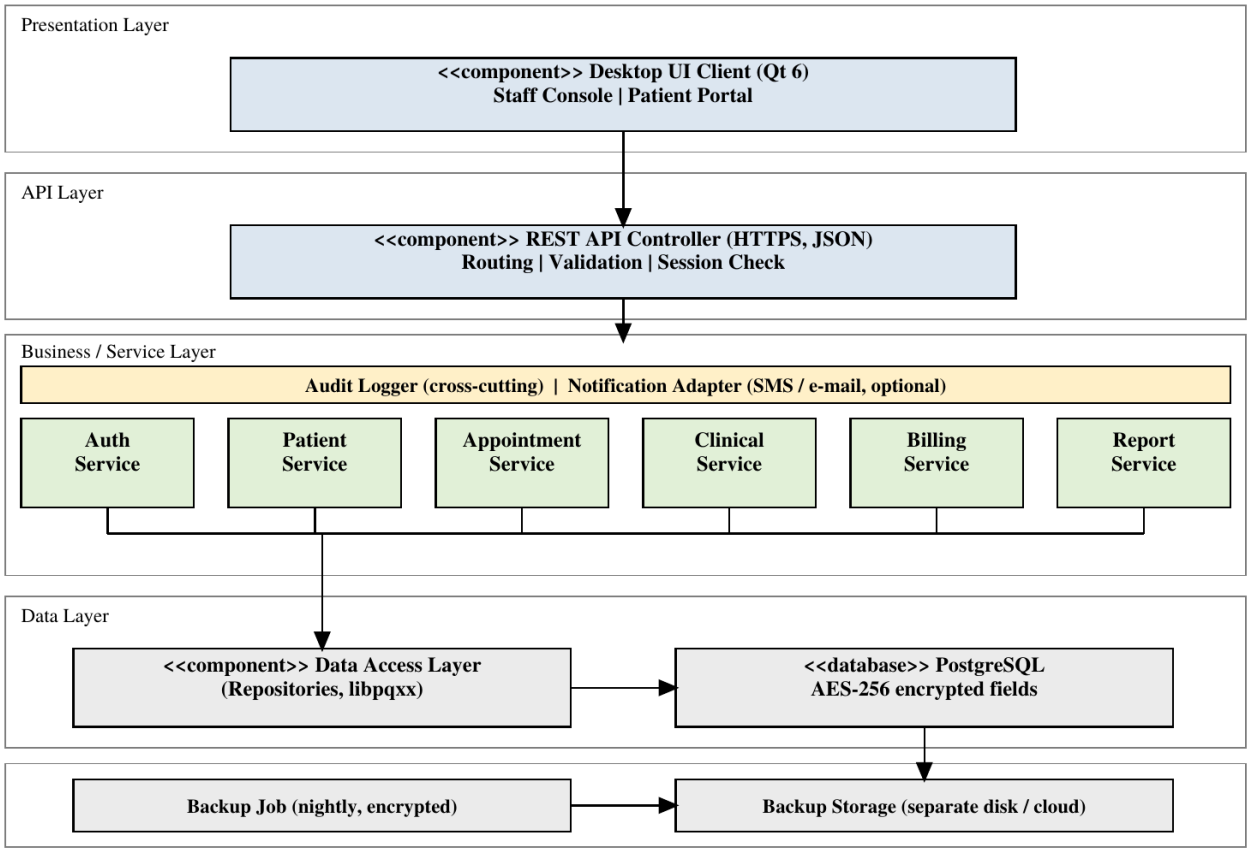


Figure 1. Component diagram of HMS (layered architecture). Arrows show the direction of calls; upper layers call only the layer directly below.

3.4 Component Descriptions

Component	Responsibility
Desktop UI Client (Qt 6)	Staff console and patient portal; collects input, shows results; holds no patient data after logout.
REST API Controller	Terminates HTTPS, routes requests, validates input and session token, checks the role, converts errors to the standard error format.
Auth Service	Login, password hashing and policy, lock-out, session tokens, roles, user management (HMS-F-001 to 004, F-060).
Patient Service	Registration, validation, duplicate check, search, update with history (HMS-F-010 to 014).
Appointment Service	Slot availability, booking, double-booking prevention, reschedule/cancel rules, doctor schedule (HMS-F-020 to 024).
Clinical Service	Visits, diagnoses, prescriptions, allergy check, patient history, including a patient's view of own records (HMS-F-030 to 034).
Billing Service	Bill creation and tax calculation, payments, status, receipts; a patient's view of own bills (HMS-F-034, F-040 to 042).
Report Service	Daily appointment and revenue reports, PDF/CSV export (HMS-F-050).
Audit Logger	Append-only audit records of logins, views and changes (HMS-SEC-005, HMS-SEC-008, HMS-F-061).
Notification Adapter	Optional interface for SMS or e-mail reminders; disabled in release 1.0.
Data Access Layer	Repository classes using parameterised libpqxx queries; field-level AES-256 encryption of PHI.
PostgreSQL Database	Stores all persistent data.

Component	Responsibility
Backup Job	Nightly encrypted dump to separate storage; supports restore within 4 hours (HMS-NF-006).

3.5 Chosen Architecture Pattern and Rationale

Layered (n-tier) client-server architecture with presentation, API, service and data layers. Each layer depends only on the one below it, so services can be unit-tested with mock repositories (HMS-NF-008), security checks are concentrated in the API and service layers, and the UI can be changed without touching business rules.

ADR	Decision	Alternatives rejected and reason
ADR-1	Layered architecture with a modular monolith server	Microservices: too much deployment and network complexity for a small team and a single hospital; MVC desktop-only: no shared data for many users.
ADR-2	PostgreSQL as the database	SQLite: weak for 200 concurrent users; NoSQL: relational integrity needed for billing and appointments.
ADR-3	Clients talk to a REST API over HTTPS, never directly to the database	Direct database access: would spread credentials to every client and bypass server-side authorisation (HMS-SEC-007).
ADR-4	Field-level AES-256 encryption of PHI in the DAL	Disk encryption only: does not protect against a database dump or an over-privileged DB user.

3.6 Technology Stack and Data Stores

Concern	Choice
Language / build	C++17, CMake, GCC 11+ / MSVC 2019+
Client UI	Qt 6 Widgets
HTTP server and JSON	cpp-httplib (with OpenSSL) and nlohmann/json
Database access	PostgreSQL 15 with libpqxx
Cryptography	OpenSSL 3 (TLS, AES-256-GCM), libargon2 (password hashing)
Testing	GoogleTest, gcov/lcov, Postman/Newman, JMeter
Static analysis	clang-tidy, cppcheck

3.7 Risks and Mitigations

Risk	Mitigation
Server or network failure makes HMS unavailable	UPS, nightly backup, documented restore procedure, health-check monitoring
Memory-safety bugs in C++ code	Use RAII and smart pointers, no raw buffers, clang-tidy, sanitizers (ASan/UBSan) in CI
Slow queries with 100,000 patients	Indexes on patient name, phone, appointment (doctor, slot); load test early
Key loss or exposure	Keys kept outside source code in an OS-protected file; documented rotation

3.8 Traceability to Requirements

Component	SRS requirements realised
Auth Service	HMS-F-001, HMS-F-002, HMS-F-003, HMS-F-004, HMS-F-060, HMS-SEC-001, HMS-SEC-003, HMS-SEC-007
API Controller	HMS-F-003, HMS-NF-003, HMS-SEC-002, HMS-SEC-004, HMS-SEC-007
Patient Service	HMS-F-010, HMS-F-011, HMS-F-012, HMS-F-013, HMS-F-014, HMS-NF-001
Audit Logger	HMS-F-014, HMS-F-061, HMS-SEC-005, HMS-SEC-008
Appointment Service	HMS-F-020, HMS-F-021, HMS-F-022, HMS-F-023, HMS-F-024, HMS-NF-002
Clinical Service	HMS-F-030, HMS-F-031, HMS-F-032, HMS-F-033, HMS-F-034
Billing Service	HMS-F-040, HMS-F-041, HMS-F-042, HMS-F-034
Report Service	HMS-F-050

Component	SRS requirements realised
Data Access Layer	HMS-NF-001, HMS-NF-003, HMS-SEC-004, HMS-SEC-006
Desktop UI Client	HMS-NF-005, HMS-NF-009, HMS-NF-007
Backup Job	HMS-NF-004, HMS-NF-006
All Services	HMS-NF-008

Example: HMS-F-021 (no double booking) is realised by the Appointment Service, checked again by a unique database constraint on (doctor_id, slot_start), and verified by TC-APT-02.

3.9 Security Architecture

Trust boundaries: (1) client to server (untrusted network, protected by TLS 1.2+); (2) API Controller to services (all input validated); (3) services to database (least-privilege database account, encrypted PHI fields); (4) server to backup storage (encrypted dumps).

Threat model (STRIDE)

Threat	Example in HMS	Countermeasure	Req.
Spoofing	Attacker guesses a doctor's password or reuses a token	Argon2id hashed passwords, password policy, lock-out after 5 failures, random 256-bit session tokens, TLS	HMS-F-002, SEC-001, SEC-003
Tampering	Modified API request or SQL injection changes a bill	TLS, server-side validation, parameterised queries, least-privilege DB account, bills locked after payment	SEC-002, SEC-004
Repudiation	User denies changing a prescription	Append-only audit log with user, time, action and record ID	SEC-005, SEC-008
Information disclosure	Receptionist reads diagnoses; DB dump leaks PHI; logs expose data	RBAC and object-level checks, AES-256 field encryption, generic error messages, no PHI or passwords in logs	HMS-F-003, SEC-006, SEC-007, SEC-008
Denial of service	Flood of login or search requests	Rate limiting on login and API, connection limits and timeouts, monitoring, backups	HMS-NF-003, NF-006
Elevation of privilege	Patient calls another patient's record API	Authorisation on every API call on the server (role + ownership), no trust in UI	HMS-F-003, SEC-007

4. Design

4.1 Design Overview

Each service is a C++ class exposing a small interface and depending on repository interfaces (for example *IPatientRepository*), so that tests can inject mock repositories. Controllers translate JSON to service calls and map exceptions to the standard error format. Domain classes: *User, Patient, Doctor, Appointment, Visit, PrescriptionItem, Bill, BillItem, Payment, AuditEntry*. A request flows: Client, API Controller (token, role, validation), Service (business rules), Repository (SQL), Audit Logger.

Data design (main tables)

Table	**Key columns**
users	user_id, username, password_hash, role, active, failed_attempts, locked_until
patients / patient_history	patient_id (PID-nnnnnnnn), name*, dob*, gender, phone*, address*, allergies* (* encrypted fields); history rows keep old values
doctors	doctor_id, user_id, specialty, fee, working hours
appointments	appointment_id, patient_id, doctor_id, slot_start, status; UNIQUE (doctor_id, slot_start) for confirmed rows
visits / prescriptions	visit_id, appointment_id, symptoms*, diagnosis*, notes*; item, dosage, frequency, days, override_reason
bills / bill_items / payments	bill_id, visit_id, subtotal, tax, total, status; item, amount; payment_id, amount, mode, receipt_no
audit_log	audit_id, user_id, role, action, record_id, timestamp, source_ip (insert-only)

4.2 UML Sequence Diagrams

Seq-1: Login (HMS-F-001, F-002, F-004, SEC-001, SEC-005)

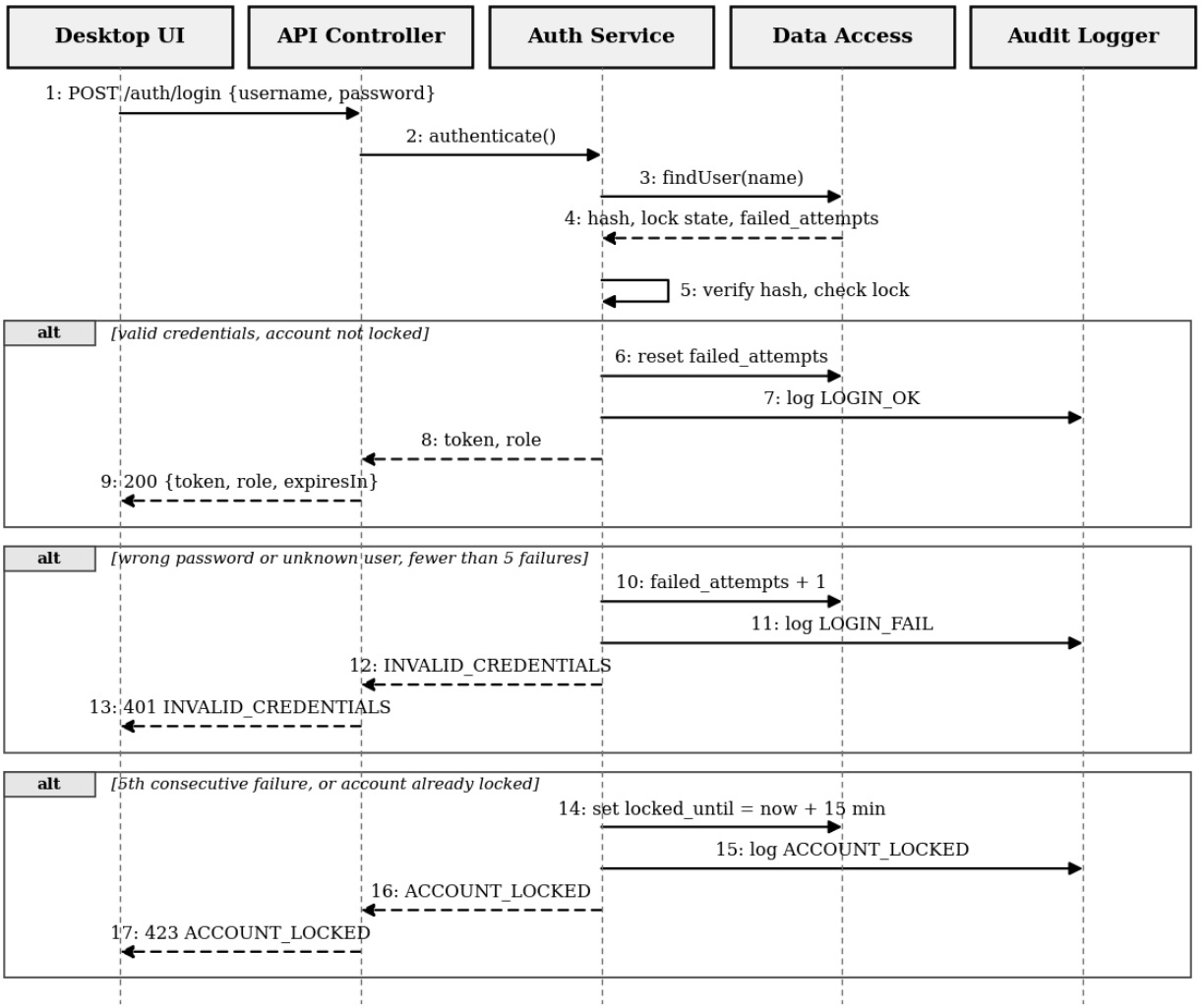


Figure 2. Login sequence with its alternatives: success (steps 6 to 9, 200 with token and role), invalid credentials (steps 10 to 13, 401 INVALID_CREDENTIALS) and lock-out after the 5th consecutive failure (steps 14 to 17, 423 ACCOUNT_LOCKED).

Seq-2: Book appointment (HMS-F-020, F-021, F-022)

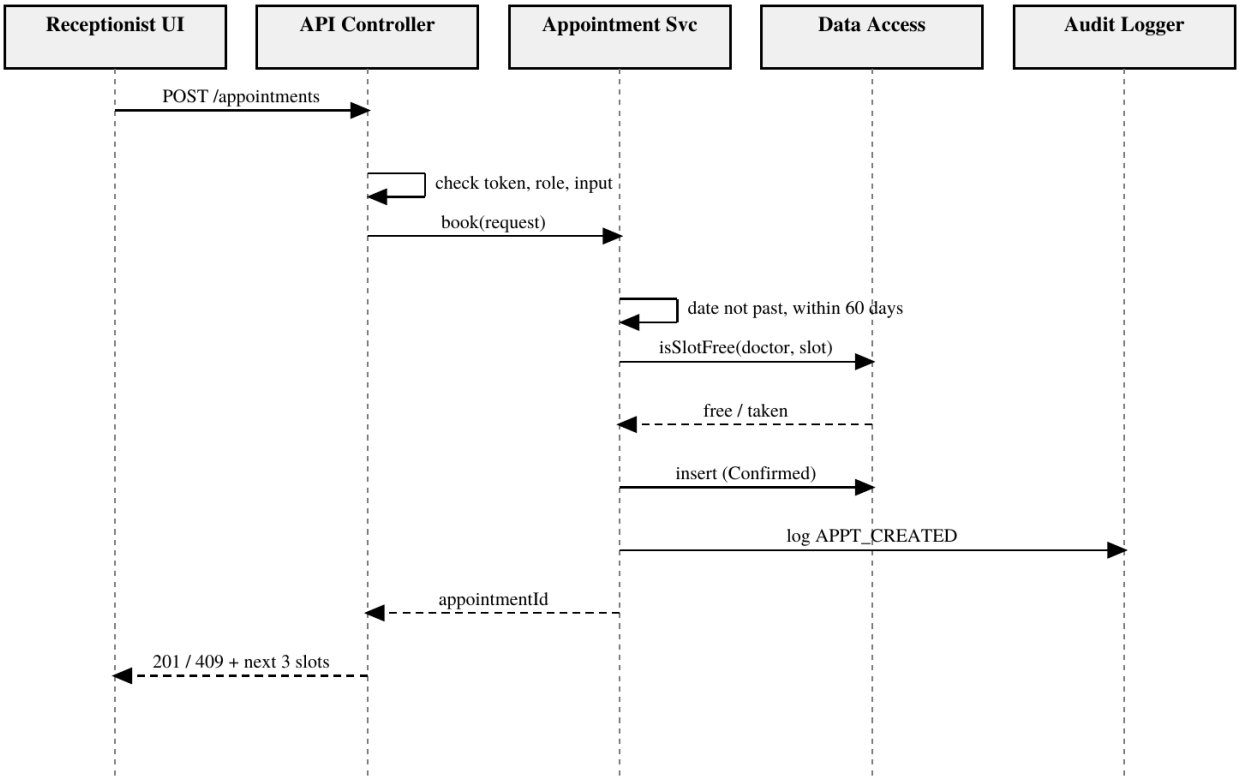


Figure 3. Booking sequence. If the slot is taken, the service returns 409 SLOT_TAKEN with the next 3 free slots; the unique constraint also stops a race between two simultaneous bookings.

Seq-3: Prescribe with allergy check (HMS-F-031, F-032)

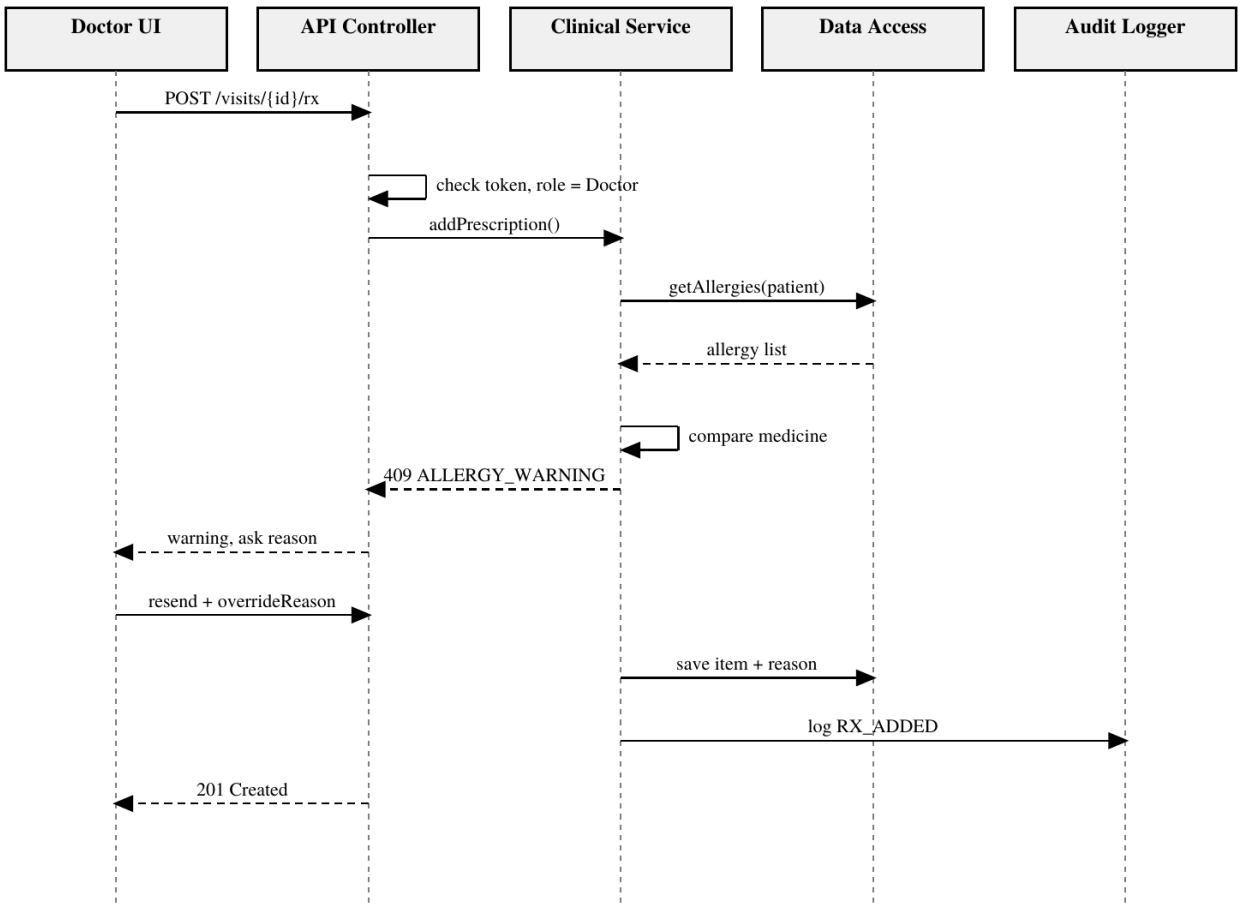


Figure 4. Prescription sequence. The first call is refused when a match is found; the second call with an override reason is saved and audited.

4.3 API Design

Base path **/api/v1**, JSON over HTTPS. Every endpoint except login needs the header *Authorization: Bearer <token>*. The API Controller checks role and ownership on each call.

Method and endpoint	Request body / params	Success response	Errors	Roles	Req.
POST /auth/login	{username, password}	200 {token, role, expiresIn}	401 INVALID_CREDENTIALS, 423 ACCOUNT_LOCKED	All	F-001, F-002, F-004
POST /auth/logout	none	204	401	All	SEC-003
POST /patients	{name, dob, gender, phone, address, allergies?}	201 {patientId}	400 VALIDATION_FAILED (field), 409 DUPLICATE_PATIENT	Receptionist	F-010 to F-012
GET /patients?query=	query: ID, name or phone	200 [{patientId, name, phone}]	400, 403	Receptionist, Doctor	F-013
PUT /patients/{id}	changed fields	200 {patient}	400, 403, 404	Receptionist	F-014
GET /doctors/{id}/slots?date=	date	200 [{slotStart, free}]	400, 404	Receptionist, Patient	F-020
POST /appointments	{patientId, doctorId, slotStart}	201 {appointmentId, status}	400 INVALID_DATE, 409 SLOT_TAKEN (+nextSlots)	Receptionist, Patient	F-020 to F-022
PUT /appointments/{id}	{action: cancel|reschedule, newSlot?}	200 {appointment}	409 TOO_LATE_TO_CHANGE, 404	Receptionist, Patient	F-023

Method and endpoint	Request body / params	Success response	Errors	Roles	Req.
GET /doctors/me/schedule ?date=	date	200 [{appointment}] sorted by time	403	Doctor	F-024
POST /visits	{appointmentId, symptoms, diagnosis, notes}	201 {visitId}	400, 403, 404	Doctor	F-030
POST /visits/{id}/prescriptions	{medicine, dosage, frequency, days, overrideReason?}	201 {itemId}	409 ALLERGY_WARNING, 400, 403	Doctor	F-031, F-032
GET /patients/{id}/history	none	200 [{visit, prescriptions}] newest first	403, 404	Doctor	F-033
GET /patients/me/history	none	200 [{visit, prescriptions}] newest first	403	Patient	F-034
GET /patients/me/bills	none	200 [{bill, payments}] newest first	403	Patient	F-034
POST /bills	{visitId, items[]}	201 {billId, subtotal, tax, total}	400, 404	Billing Officer	F-040
POST /bills/{id}/payments	{amount, mode}	201 {receiptNo, status, balance}	400 OVERPAYMENT, 404	Billing Officer	F-041, F-042
GET /reports/{type}?from =&to=&format=	type: appointments|revenue ; format: pdf|csv	200 file	400, 403	Administrator, Billing Officer	F-050
POST /users, PUT /users/{id}	{username, role, active}	201 / 200 {user}	400, 403, 409	Administrator	F-060
GET /audit?user=&from= &to=&action=	filters	200 [{auditEntry}]	403	Administrator	F-061

Example: POST /api/v1/appointments

Request	{ "patientId": "PID-00001234", "doctorId": 7, "slotStart": "2026-10-20T10:00" }
Response 201	{ "appointmentId": "APT-000981", "status": "Confirmed" }
Response 409	{ "error": { "code": "SLOT_TAKEN", "message": "This slot is already booked.", "nextSlots": ["10:30", "11:00", "11:30"] } }

4.4 Error Handling, Logging and Monitoring

All errors use one JSON format: *{ "error": { "code", "message", "field?", "requestId" } }*. The message is plain language and never exposes SQL, stack traces or file paths; the requestId links the response to the server log.

HTTP	Code	When
400	VALIDATION_FAILED / INVALID_DATE / OVERPAYMENT	Input fails validation (field named in "field")
401	INVALID_CREDENTIALS / SESSION_EXPIRED	Wrong login or missing, expired or logged-out token
403	ACCESS_DENIED	Role or ownership check fails
404	NOT_FOUND	Patient, appointment, visit or bill does not exist
409	SLOT_TAKEN / DUPLICATE_PATIENT / ALLERGY_WARNING / TOO_LATE_TO_CHANGE	Business rule conflict
423	ACCOUNT_LOCKED	5 failed logins, locked for 15 minutes
429	TOO_MANY_REQUESTS	Rate limit exceeded

HTTP	Code	When
500	INTERNAL_ERROR	Unexpected failure; details only in server log

- **Handling:** services throw typed exceptions (ValidationError, ConflictError, AuthError); the controller maps them to the table above; database transactions are rolled back on any error.
- **Logging:** application log (level, time, requestId, component, message) and a separate audit log. Neither stores passwords, tokens or clinical content (HMS-SEC-005).
- **Monitoring:** health-check endpoint every minute (availability, HMS-NF-004), request failure rate, 95th percentile latency, failed-login count, backup success/failure alert.

4.5 UX Design

- Left-hand menu shows only the functions allowed for the logged-in role.
- Patient search is always one click away (Ctrl+F) and shows results as you type.
- Booking screen shows free slots as a grid so that a receptionist books in three steps (HMS-NF-005).
- Clinical screens always show patient name, ID, age and allergies in a banner.
- Large fonts, high contrast, keyboard-only operation, clear error messages next to fields.

4.6 Open Issues and Next Steps

- How patients receive their first login (in-person activation versus OTP) is to be decided with the hospital.
- Drug-allergy matching uses name matching only; a drug-class database is a possible enhancement.
- Future enhancements: SMS reminders, laboratory integration, mobile patient app, biometric login.
- Part-2: implement services and repositories, write unit tests, run the test cases of the Test Plan.

5. Appendices

5.1 Glossary: see SRS Appendix A and Section 1.4. **5.2 References:** IEEE 42010, IEEE 1016, OWASP ASVS 4.0, NIST SP 800-160, DPDP Act 2023, HMS SRS v1.0, HMS Test Plan v1.0. **5.3 Tools:** draw.io / PlantUML for diagrams, Postman and Swagger/OpenAPI for the API, CMake, clang-tidy, GoogleTest.